PlotTwist

Dossier de lancement — Rendu R1

Nourane Hammouda · Chaïma Mabrouki · Ornela Mothu · Zaineb BenMahmoud

20 octobre 2026

Formation : Master 1 MIAGE · année universitaire 2026-2027
Version du document : 1.1 · **Version du projet :** phase de lancement
Dépôt principal : https://github.com/nourane-hammouda/PlotTwist

1. Présentation de l'équipe

PlotTwist est réalisé par une équipe de **quatre étudiantes** de Master 1 MIAGE. Aucun binôme ni étudiant isolé n'est laissé de côté.

Membre	N° étudiant	Compte GitHub	Adresse de commit
Nourane Hammouda	43017567	nourane-hammouda	`nouranehamouda09@gmail.com`
Chaïma Mabrouki	43001959	Chaimambk	`chaima.mbk21@gmail.com`
Ornela Mothu	43009011	ornelamt	`ornelamothu4@gmail.com`
Zaineb BenMahmoud	43014883	zainebbmn	`zeyneb.b.mah04@gmail.com`

Chaque membre utilisera **son propre compte développeur**. Les commits seront réalisés avec le nom et l'e-mail ci-dessus, afin que les contributions individuelles soient identifiables à l'évaluation et à la soutenance.

Le dépôt principal est celui du **back Java**. Il accueillera le code métier, l'API REST, le dossier (`/doc`), la documentation et la CI. Le front sera développé **dans le même dépôt**, dans un module distinct. La répartition des axes fonctionnels n'est **pas encore arrêtée** : elle le sera avant le démarrage de la v0.1.0, à raison d'un axe par personne.

2. Présentation générale du projet

PlotTwist est une application de **génération et de résolution d'enquêtes interactives**.

À partir de personnages, de lieux, de relations et d'événements, le système construit une enquête structurée : suspects, victime, coupable, mobile, chronologie, déplacements, témoignages, vrais indices et fausses pistes. Le joueur enquête (lieux, indices, interrogatoires) puis accuse (coupable, mobile, preuves).

Ce n'est pas une bibliothèque d'histoires écrites à l'avance. Le cœur du produit est un **moteur qui génère, contrôle et fait évoluer une enquête cohérente** selon des contraintes logiques. Le proof of concept reste compact : 4 à 6 suspects, 4 à 6 lieux, 8 à 12 indices, une partie d'une quinzaine de minutes, des dialogues structurés (intentions prédéfinies, pas un chatbot libre).

3. Caractère innovant

L'innovation repose sur la **génération procédurale d'enquêtes résolubles**. Chaque partie peut produire une combinaison différente de personnages, relations, chronologies, indices et fausses pistes.

Le moteur devra garantir notamment que le coupable a un mobile plausible et la possibilité matérielle d'agir ; que les déplacements sont compatibles avec la chronologie et le graphe des lieux ; qu'un personnage n'est pas simultanément dans deux lieux ; que plusieurs suspects restent plausibles en début de partie ; et que des indices décisifs permettent d'identifier le coupable sans rendre l'enquête triviale ni impossible.

La dimension algorithmique porte sur la **génération sous contraintes**, les **graphes** (relations, lieux, plus courts chemins), la **cohérence temporelle**, la **détection de contradictions** et le **classement des suspects**. Ce n'est pas une surcouche CRUD : chaque feature majeure encapsule un algorithme nommé, testable, avec invariants et cas limites.

4. Scénarios d'usage prévus

Utilisateurs visés. Joueurs d'enquêtes courtes. Le professeur doit pouvoir lancer une partie en soutenance et tenter de la résoudre.

Scénario principal — générer, enquêter, accuser.

1. Le joueur démarre une partie (paramètres simples : nombre de suspects, difficulté, nombre de lieux).
2. Le back génère l'enquête (personnages et relations, coupable et mobile, chronologie, indices et fausses pistes) puis vérifie cohérence et résolubilité.
3. Le joueur explore les lieux, collecte des indices, interroge les suspects sur des thèmes structurés (alibi, relation avec la victime, présence dans un lieu, confrontation avec un indice).
4. Il formule une accusation. L'application compare la proposition à la solution et produit un bilan.

Scénarios secondaires. Aide adaptée à l'avancement (sans révéler immédiatement le coupable) ; détection d'une contradiction entre témoignage, chronologie et temps de déplacement ; relance d'une nouvelle enquête.

5. Fonctionnalités envisagées et progression

Le développement s'organise autour de **quatre axes**, qui seront attribués **un par membre** avant la v0.1.0. Chaque axe évolue sur trois itérations (une feature majeure par personne et par version). Cette trajectoire n'est pas un engagement à livrer douze modules de taille identique : le périmètre pourra être ajusté.

Axe	v0.1.0	v0.2.0	v1.0.0
Génération de l'enquête	Personnages, relations et coupable	Sélection contrainte du coupable et du mobile	Validation globale et régénération
Chronologie et déplacements	Emplois du temps cohérents	Graphe des lieux et calcul des déplacements	Détection d'incohérences spatio-temporelles
Indices et déduction	Génération et placement des indices	Moteur de déduction (plausibilité)	Aide adaptative
Interactions et résolution	Accusation et vérification	Interrogatoire structuré	Score et bilan explicatif

v0.1.0 (24 novembre). Boucle jouable minimale : génération, chronologie, indices, accusation, vérification. Une feature majeure de bout en bout par membre, README et badges, CI.

v0.2.0 (8 décembre). Approfondissement logique (contraintes, déplacements, déduction, interrogatoires) et **conteneurisation**.

v1.0.0 (19 janvier). Résolubilité, contradictions, aide, scoring, stabilisation des tests et de l'interface, soutenance.

6. Architecture technique

Architecture 3 tiers : **front → API REST → back → base de données**. Le front n'accède jamais directement à la BD.

```
Front (web, éventuellement CLI)
        │  HTTP / JSON
API REST Spring Boot (controllers, DTO, OpenAPI)
        │  appels métier
```

```
Back Java 21 (services + algorithmes purs, JGraphT)
        │  JPA / Hibernate
PostgreSQL (schéma versionné par Flyway)
```

Front. Création de partie, suspects, lieux, carnet d'indices, interrogatoires, accusation, bilan. Framework non figé (React, Vue.js ou Thymeleaf). Une CLI pourra servir de premier client. Le front n'intervient **qu'après** back testé et API exposée.

API REST. Versionnée sous `/api/v1` : `/investigations`, `/characters`, `/locations`, `/clues`, `/interrogations`, `/hints`, `/accusations`, `/result`. OpenAPI / Swagger, codes HTTP corrects, erreurs structurées, validation Jakarta.

Back. Java 21 et Spring Boot 3. La logique (génération, contraintes, chronologie, graphes, déplacements, indices, déduction, interrogatoire, accusation, score) vit dans des **classes Java pures**, indépendantes du framework. Les contrôleurs REST ne portent pas la logique. JGraphT servira aux graphes social et spatial (Dijkstra / A*).

Base de données. PostgreSQL via Spring Data JPA / Hibernate. Entités pressenties : `Investigation`, `GameSession`, `Character`, `Relationship`, `Location`, `Connection`, `TimelineEvent`, `Motive`, `Clue`, `Interrogation`, `Testimony`, `Accusation`, `GameResult`. Migrations **Flyway** (une migration poussée n'est jamais réécrite). Mapping entités ↔ DTO par records Java et, si besoin, MapStruct.

Stack envisagée

Besoin	Choix	Rôle
Langage / build	Java 21, Maven	LTS, écosystème Spring
Backend / API	Spring Boot 3, Spring Web	REST, DI, erreurs, validation
Données	PostgreSQL, Spring Data JPA, Hibernate, Flyway	Persistance et schéma versionné
Graphes	JGraphT	Relations, lieux, plus courts chemins
Tests	JUnit 5, Mockito, AssertJ, Testcontainers	Unitaire et intégration PostgreSQL
Qualité	JaCoCo, SpotBugs ou Error Prone	Couverture et défauts
Livraison	Docker Compose, GitHub Actions	Conteneurs et CI/CD

Chaque feature : tests unitaires de l'algorithme (nominaux, limites, invariants). Tests d'intégration (persistance, API) avec Testcontainers. CI : compilation, tests, couverture, analyse à chaque push et pull request. Docker (back, PostgreSQL, front) pour la v0.2.0.

Git. Une branche par feature, PR relue par un autre membre, `main` pour les versions taguées. Tags de pré-livraison en fin de séance (`v0.1.0-SNAPSHOT`, etc.).

7. Organisation des contributions

Les membres ne seront **pas** répartis par couche technique (une personne BD, une personne front). Chaque membre portera un **moteur métier de bout en bout**, une fois les axes attribués.

Démarche d'une feature : (1) idéation dans une issue (besoin, algorithme, complexité, schéma, critères d'acceptation) ; (2) back (migration, modèle, service, algorithme en classe Java pure) ; (3) tests unitaires a minima ; (4) API REST documentée ; (5) **front en dernier**.

Suivi par commits, branches, pull requests, issues et tests. Chaque membre dépose individuellement chaque rendu dans l'espace devoir.

8. Protocole d'utilisation de l'intelligence artificielle

L'IA générative est un **outil d'assistance**. Elle ne remplace ni la conception ni la compréhension. En soutenance, chacune doit pouvoir expliquer et refaire au tableau ses features.

Autorisé. Pistes de conception, reformulation de documentation, explication d'erreur, structure de tests, données fictives, comparaison d'approches, relecture de code déjà compris.

Interdit. Intégrer du code non compris ; accepter une proposition sans vérification ; laisser l'IA décider de l'architecture ou de la cohérence d'une enquête ; livrer des sorties non testées ; transmettre secrets ou jetons.

Validation. Relu, expliqué, adapté, testé, contrôlé, puis examiné en pull request. La cohérence de l'enquête reste **déterministe et testable**. Les usages significatifs sont tracés dans la PR (outil, objectif, repris / modifié / rejeté).

9. Dépôts du projet

Dépôt principal : https://github.com/nourane-hammouda/PlotTwist

```
/
├── doc/          dossier-lancement.md / .pdf
├── src/
├── pom.xml
├── README.md
└── .github/workflows/
```

Front : intégré au dépôt principal. Pas de second dépôt à ce stade.

10. Conclusion et perspectives

PlotTwist vise un proof of concept fondé sur une **logique métier algorithmique** : générer une enquête cohérente, vérifier les contraintes spatio-temporelles, raisonner à partir des indices, évaluer l'accusation.

Prochaines étapes : attribution des quatre axes ; invitation des collaboratrices au dépôt ; configuration Git des identités de commit ; squelette Spring Boot / Maven ; issues d'idéation de la v0.1.0. La démonstration de soutenance consistera à générer une enquête et à la jouer.